Why a Fixed-Size Memory Can't Keep All Your Secrets

DataForge 2026 Pathway Track — Blog Post | Topic #5: Fixed-Size Recurrent State vs. Key-Value Caching

If you've used a long-context chatbot, you've relied on a Transformer's key-value (KV) cache — a growing table of every token it has seen. Ask it to quote something from ten thousand tokens ago, and it usually can. That's not a soft "attention is powerful" story. It's an architectural fact you can prove, and it puts a hard ceiling on an entire family of newer, faster models.

Two ways to remember a sequence

A Transformer compares token *i* against every key and value stored so far — the KV cache. That cache grows linearly with sequence length: twice the input, twice the memory. Expensive, but nothing is thrown away.

The alternative — state space models (SSMs) like Mamba — compress everything seen so far into one fixed-size vector, updated and discarded token by token. Memory stays flat regardless of length, which is why these models are prized for cheap, fast inference on long inputs.

The mechanism, not just the symptom

Jelassi, Brandfonbrener, Kakade, and Malach make the limit precise. They define "generalized state space models" (GSSMs) as any architecture with a fixed-size state space, and prove such a model cannot copy an input sequence unless its state holds nearly one bit of memory per bit of input — otherwise it fails on more than half of all possible sequences (Jelassi et al., 2024). A Transformer faces no such ceiling: the authors construct a two-layer Transformer that copies sequences exponentially longer than its own head count, by hashing n-grams into keys and looking up what came after them before.

This is a mechanism, not an observed slowdown: a GSSM forgets not from bad training, but because compressing an unbounded stream into a bounded vector necessarily overwrites old information once input exceeds capacity.

Experiments back the theory. Training Transformers and GSSMs (including Mamba) at matched size (~160M parameters), Transformers needed roughly 100x fewer examples to learn to copy length-300 strings, and generalized far better to unseen lengths, while GSSM accuracy collapsed almost immediately past training length (Jelassi et al., 2024). At pretrained scale, a 410M-parameter Transformer beat a 2.8B-parameter Mamba model at a "phone book" lookup task once the book held 70+ entries — an eight-times-smaller model winning on architecture alone (Jelassi et al., 2024).

Mamba's answer: make the fixed state smarter

Fixed-state models trade exhaustive recall for efficiency, and the interesting research shrinks that trade-off. Mamba, from Gu and Dao, keeps the state fixed-size but makes the update rule input-dependent: how much of each token to write in, and how fast old state decays, are computed from the input itself (Gu and Dao, 2023). That "selection mechanism" is a policy for what's worth keeping under a bounded budget, narrowing the language-modeling gap with Transformers while

keeping linear-time, constant-memory inference. It doesn't remove Jelassi et al.'s ceiling: a smarter update rule still can't store more bits than the state has room for. Mamba just makes better use of the bits it has.

Where Hebbian memory and BDH fit

Fixed-size, bounded-capacity state predates modern SSMs. Classical Hopfield networks store patterns in an N×N weight matrix whose capacity is a *fixed fraction* of neuron count (roughly 14%, tightened by Stojnic, 2024) — same underlying reason: a bounded store can't hold unboundedly many independent patterns without interference. Pathway's Dragon Hatchling (BDH) faces a related fixed-capacity tradeoff, but per its primary technical report replaces a purely fixed rule with a Hebbian-updated fast-weight state that sits on top of separately gradient-trained parameters — changing *how* the budget is spent and *who set the rule*, not whether a budget exists. We flag this because it bears directly on the mechanism above, not to claim BDH's own benchmark numbers, which are outside this post's scope.

The open question

The two failure modes differ in kind: a Transformer's context window is a hard cutoff; a fixed-state model's forgetting is soft interference, where older information degrades as new information arrives. That's why fixed-state models degrade gracefully on compressible natural language but fail sharply on tasks built to defeat compression, like copying shuffled text (Jelassi et al., 2024). The live question is whether a hybrid — fixed-size state plus a small, selective side-cache for the few tokens that truly need exact recall — can get GSSM-level cost without the full copying penalty. That's an active design space, not a settled answer.

Limitation to flag: these copying results are strongest on adversarial data built to resist compression; Jelassi et al. themselves show the gap narrows (though doesn't close) on natural, compressible language. If your use case never needs exact long-range recall, this trade-off may not bite you.

References:
Jelassi, S., Brandfonbrener, D., Kakade, S. M., & Malach, E. (2024). Repeat After Me: Transformers are Better than State Space Models at Copying. arXiv:2402.01032.
Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
Stojnic, M. (2024). Capacity of the Hebbian-Hopfield network associative memory. arXiv:2403.01907. (Cited for the fixed-capacity-fraction framing; not a primary source for the KV-cache argument above.)
Dragon Hatchling (BDH) — Pathway AI primary technical report. (Verify all BDH-specific claims against the primary source before submission.)